PRINCETON UNIVERSITY PRESS

Chapter Title: D LATTICE COMPUTATION

Book Title: Natural Complexity

Book Subtitle: A Modeling Handbook

Book Author(s): Paul Charbonneau

Published by: Princeton University Press

Stable URL: https://www.jstor.org/stable/j.ctt1vwmgzt.19

JSTOR is a not-for-profit service that helps scholars, researchers, and students discover, use, and build upon a wide range of content in a trusted digital archive. We use information technology and tools to increase productivity and facilitate new forms of scholarship. For more information about JSTOR, please contact support@jstor.org.

Your use of the JSTOR archive indicates your acceptance of the Terms & Conditions of Use, available at https://about.jstor.org/terms

Princeton University Press is collaborating with JSTOR to digitize, preserve and extend access to *Natural Complexity*

JSTOR

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

D

LATTICE COMPUTATION

Most computational implementations of the complex systems explored in this book are defined over lattices: sets of interconnected nodes on which the dynamical variables of the problem are represented. There are two interconnected concepts that need to be distinguished: lattice *geometry* and *connectivity*. Geometry is set by the relative positions of lattice nodes in physical space; connectivity refers to the coupling between nodes, i.e., which neighboring nodes interact with any given node. The foregoing discussion is framed in the context of 2-D lattices, but generalization to higher dimensions is usually straightforward.

When carrying our numerical simulations on lattices, nodal values are usually stored as arrays in the computer's memory, having the same dimension and lengths as the said lattices, i.e., nodal values on a 128×128 lattice are stored in a 2-D array having length 128 in each dimension. The syntax for defining such arrays is described in appendix A.2. From the user's point of view (but not in the computer's RAM), 2-D arrays are thus defined in terms of rows (first array dimension) and columns (second dimension), as with a matrix, which effectively represents a form of Cartesian geometry. Storing, accessing, and plotting nodal values for a Cartesian lattice is thus algorithmically trivial.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

Lattice geometries other than Cartesian can still be stored in 2-D arrays, by a suitable choice of connectivity. The idea has already been illustrated in panels C and D of figure 2.5: with appropriate horizontal shifting of nodal positions, a 2-D Cartesian lattice with anisotropic 6-neighbor connectivity can be reinterpreted as a triangular lattice with isotropic 6-neighbor connectivity. Geometry is secondary (except when plotting!), and connectivity is the key.

D.1 Nearest-Neighbor Templates

Setting connectivity for a lattice is best accomplished using a *nearest-neighbor template*. This gives the *relative* positions of nearest neighbors with respect to a given nodal position (i, j). For example, on a Cartesian lattice the 4 nearest neighbors of a node (i, j) are located at the bottom, right, top, and left; or, in terms of nodal numbering, $(i + 1, j)$, $(i, j + 1)$, $(i - 1, j)$, and $(i, j - 1)$. This can be stored in two 1-D integer arrays of length 4, one for each lattice dimension. Under Python/NumPy this is initialized as follows:

```
dx=np.array([1,0,-1,0])
dy=np.array([0,1,0,-1])
```

This is known as the *von Neumann neighborhood*. Including the 4 diagonal neighbors yields the *Moore neighborhood*, which would be defined by the template arrays

```
dx=np.array([0,1,0,-1,-1,1,1,-1])
dy=np.array([-1,0,1,0,-1,-1,1,1])
```

For the 6-neighbor triangular lattice of figure 2.5, the template arrays are

```
dx=np.array([0,1,0,-1,-1,1])
dy=np.array([-1,0,1,0,1,-1])
```

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

Whatever the connectivity, lattice operations can use these template arrays to efficiently retrieve nearest-neighbor information. For example, with nodal values stored in a 2-D array named `grid`, calculating the sum of nodal values for all nearest neighbors of node (i, j) under the von Neumann neighborhood could be coded like so:

```
sum_nn=0.
for k in range(0,4): sum_nn+=grid[i+dx[k],j+dy[k]]
```

or equivalently, by invoking the `sum()` function from Python's NumPy library:

```
sum_nn=np.sum(grid[i+dx[:],j+dy[:]])
```

These instructions would be typically embedded within two loops for the indices i and j, thus scanning all lattice nodes. There is one pitfall to this strategy: as shown in figure D.1, still for the von Neumann neighborhood, it will fail for nodes at the boundaries of the lattice, which have only 3 nearest neighbors (and only 2 for the four corner nodes).

There are ways out of this difficulty, of course. In principle, the most straightforward is to treat boundary nodes separately, for example through the use of suitably modified template arrays used only for boundary nodes. However, this leads to cumbersome extra coding that significantly lengthens a simulation code and reduces its readability. A better strategy is to make use of *ghost nodes*, as shown in figure D.2. The 10×10 lattice of figure D.1 is now padded on all sides with a layer of additional nodes (open gray circles). This expanded lattice is now of size 12×12, but computations associated with the model's dynamical rules take place only in the interior 10×10 block of nodes corresponding to the original, unpadded lattice. Unlike in figure D.1, using the 4-neighbor template on the red node, now numbered

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

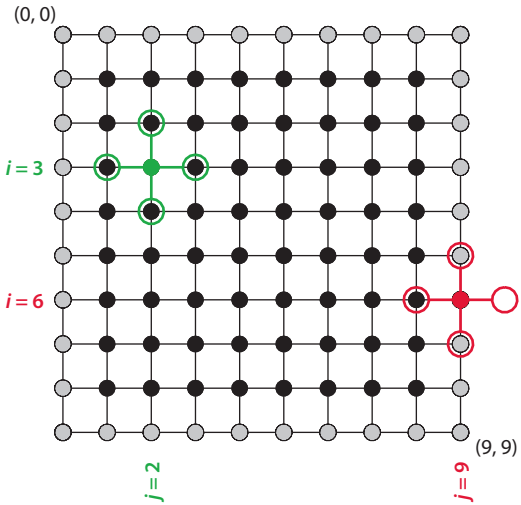

Figure D.1. Use (green) and misuse (red) of a 4-nearest-neighbor template on a 10×10 Cartesian lattice. Nodes are numbered by a pair of indices (i, j), starting at the top-left corner and increasing downward and to the right. The template functions well for interior nodes (black) but will lead to out-of-bounds array indexing for boundary nodes (gray) unless alternative, reduced template arrays are introduced for boundary nodes, or additional conditional instructions (**if**. . . **else**) are added within the code.

$(i, j) = (7, 10)$, will not exceed the array length in the horizontal since $j + 1 = 11$, which is now legal.[1]

Which numerical value is to be assigned to ghost nodes is dependent on the boundary conditions of the problem. In the earthquake model of chapter 8, for example, the ghost nodes are

[1]Some computing languages allow the use of negative integers to index array elements, so, for example, here each dimension of the 12×12 lattice could have the nodes numbered from -1 to 10, so that the green and red nodes retain their original numbering $(i, j) = (3, 2)$ and $(6, 9)$, as in figure D.1. I stayed away from this cleaner numbering strategy for reasons of portability to languages that do not allow such generalized array indexing.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

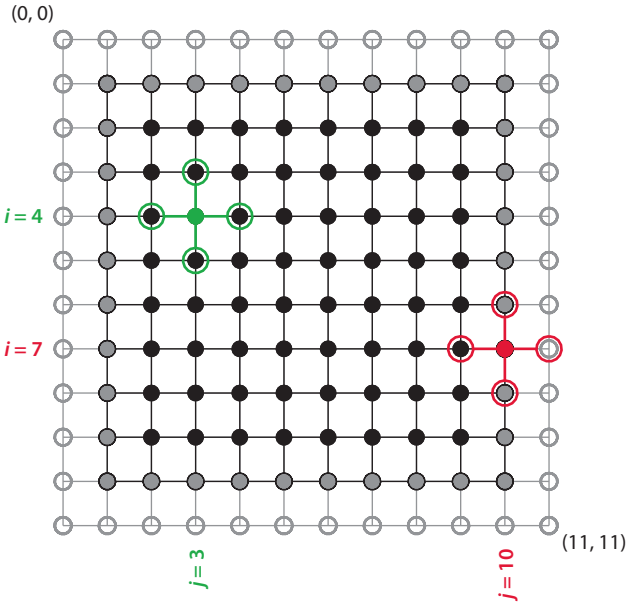

Figure D.2. The same lattice as in figure D.1, but now surrounded by a layer of ghost nodes (open gray circles). The full lattice is now of dimensions 12×12, and the 4-neighbor template can be used on all nodes of the embedded original 10×10 lattice even for the (true) boundary nodes (solid gray circles).

simply set to zero and retain that value throughout the whole simulation. This is as easy as it gets. In the hodgepodge machine simulations of chapter 11, on the other hand, the ghost nodes are used to enforce *periodic boundary conditions*, as detailed in the following section.

D.2 Periodic Boundary Conditions

In some lattice-based models introduced in this book, periodic boundary conditions are imposed on the lattice. Sometimes this is dictated by the geometry of the problem. Consider, for example, ants walking on the surface of a sphere. Using

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

latitude–longitude coordinates, an ant walking eastward and crossing longitude 360° must instantly "reappear" at longitude 0° because both correspond to the same point on the sphere. Longitudinal periodicity is then mandatory. Now imagine ants walking on the surface of a torus. Longitude is again periodic, but now, so is the latitudinal direction, since an ant starting in the equatorial plane and walking north will travel a circular path that will bring it back to its starting point. When simulating such a walk, the torus can thus be mapped to a square with periodicity enforced both horizontally and vertically. This is the geometric interpretation to be ascribed to the highway-building ant of section 2.4, and to the flocking simulation of chapter 10.

In other instances, periodic boundary conditions are used simply because we cannot specify boundary values, and doing so arbitrarily would perturb the evolution of the system. This is the case with the hodgepodge machine simulations of chapter 11. Enforcing periodic boundary conditions then implies that the simulated domain is but a "tile" that repeats itself across space to infinity, exactly like on a tiled floor (made of identical tiles, and without the infinity part). In such a situation, we must simply accept the fact that the simulation cannot generate or accommodate structures that have length scales larger than the simulated periodic unit.

Figure D.3 shows how ghost nodes can be used to enforce periodic boundary conditions. The true boundary nodes of our now familiar original 10×10 lattice have been colored according to the side they belong to, with four more distinct colors used for the corner nodes. If the original lattice were to be replicated horizontally and vertically to tile the whole space under the assumption of periodicity (as for the unit square domain in figure 10.4), then the top row of ghost nodes (open blue circles plus corner nodes) are really the "same" nodes as the bottom row of the original 10×10 lattice (solid blue nodes plus corner nodes, boxed in blue). Vertical periodicity can therefore be enforced by

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

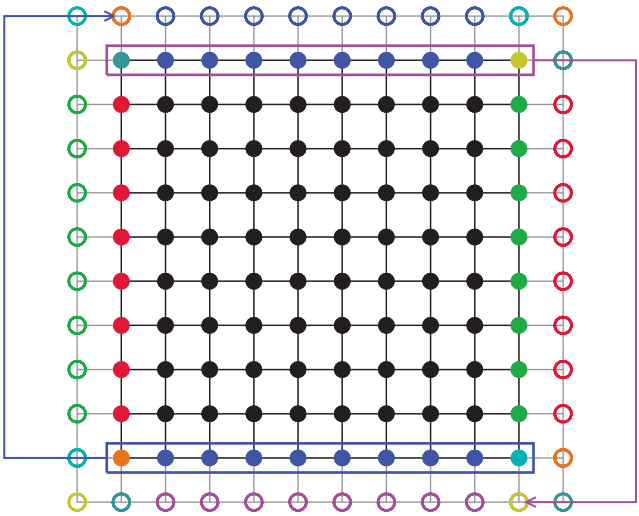

Figure D.3. Enforcing periodic boundary conditions via ghost nodes. The boundary nodes of the original 10×10 lattice (solid gray nodes in figure D.1) are copied to the ghost node layer (open circles) on the opposite side of the enlarged 12×12 lattice, following the color coding given. Note how the corner nodes of the 10×10 lattice get copied into three distinct ghost nodes. Interior nodes (in solid black) remain unaffected by this whole procedure.

copying the 10 nodal values of this row (boxed in blue) to the corresponding nodes of the top row of ghost nodes, as indicated by the blue arrow. The same applies to the top row of the original 10×10 lattice (purple), which gets copied into the bottom row of ghost nodes. The same procedure is used in the horizontal direction, as indicated by the color coding on the figure. Note how each corner node in the 10×10 lattice gets copied thrice into ghost nodes: once horizontally, once vertically, and once diagonally to the opposite corner of the 12×12 lattice. The function `periodic`, given within the code in figure 11.4, gives a compact algorithmic implementation of this procedure.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

Periodicity in one spatial dimension amounts to assuming that the 1-D domain is a closed ring, which is much easier to implement; see, for example, the 1-D CA code of figure 2.4.

D.3 Random Walks on Lattices

Random walks can be defined over a lattice, with walkers constrained to move from one node to a randomly selected nearest-neighbor node, according to some suitably defined neighbor template. All that is needed is to generate a random integer to pick an element of the appropriate template arrays. The following code fragment shows how to set up a random walk of N steps, here on a 2-D Cartesian lattice with 4-neighbor connectivity, and with the walker starting at the (arbitrary) nodal position $(i, j) = (5, 5)$ on the lattice:

```
import numpy as np
N=100                              # number of random walk steps
dx=np.array([0,1,0,-1])            # template arrays for 4 neighbors
dy=np.array([-1,0,1,0])
...
i,j=5,5                            # initial nodal position of walker
for  k in range(0,N):              # walk N steps
    r=np.random.choice([0,1,2,3])  # random integer between 0 and 3
    i+=dx[r]                       # take one random step
    j+=dy[r]
...
```

An example is shown in figure D.4. The first 18 steps of the walk are shown, and reveal some occasional backtracking (steps 2–3–4 and 13–14–15). Note also that, after the sixth step, the walker is actually back at its starting position.

This may appear to be a strongly constrained type of random walk, but when simulating many such random walks on the lattice over a great many steps, the orientation of the displacement vector is effectively random, and its ensemble averages $\langle \boldsymbol{D} \rangle_n = 0$, and $\left\langle D^2 \right\rangle \propto n$, just like in a classical random walk (see section C.5).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

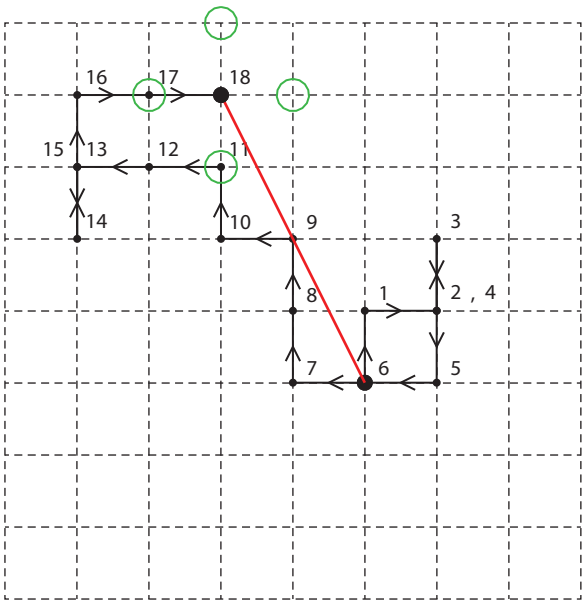

Figure D.4. The first 18 steps in a random walk on a 2-D Cartesian lattice with 4-neighbor connectivity, starting at the lattice center. Successive nodal positions are numbered, and the thick red line segment indicates the displacement vector $\boldsymbol{D}_n$ at step 18. The 19th step will land the walker on one of the four nodes circled in green, which one being chosen randomly, in a manner independent of the current position or direction of past steps (see text). Compare to figure C.3.

Moreover, on length scales much larger than the internodal distance, the spatial distributions of walkers are essentially the same in both cases. Figure D.5 illustrates this, now for four 400-step random walks on a larger lattice, still with 4-neighbor connectivity. For displacements much larger than the microscopic scale set by the internodal distance, the spatial distribution of end points is statistically undistinguishable from that associated with a conventional 2-D random walk (cf. figure C.4).

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

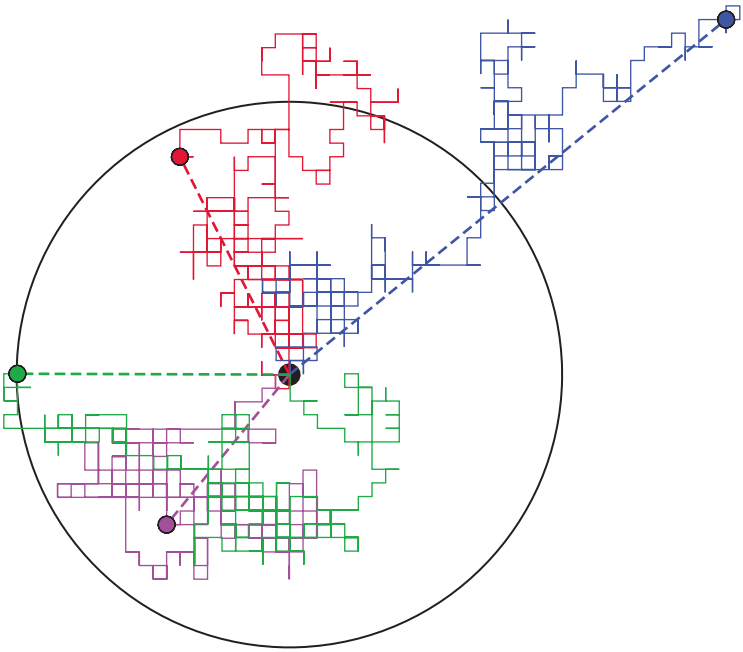

Figure D.5. Four 400-step random walks on a lattice with 4-neighbor connectivity. The starting point is indicated by a black dot at the center, and the circle indicates a displacement $D \equiv \sqrt{\boldsymbol{D} \cdot \boldsymbol{D}} = \sqrt{n} = \sqrt{400} = 20$. Compare to figure C.4.

Random walks on lattices become particularly useful when simulating a system where many walkers moving simultaneously on the lattice interact locally in some way (e.g., the healthy and sick agents in the epidemic propagation simulations of chapter 9). Knowing the position (i, j) of a walker on the lattice, only this node (or nearest-neighbor nodes) must be checked for the presence of another walker. In a classical random walk, this would involve instead the calculation of $\simeq N^2/2$ pairwise distances, to pick which walkers are within some set distance inside which the interaction takes place (as for the repulsion and

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

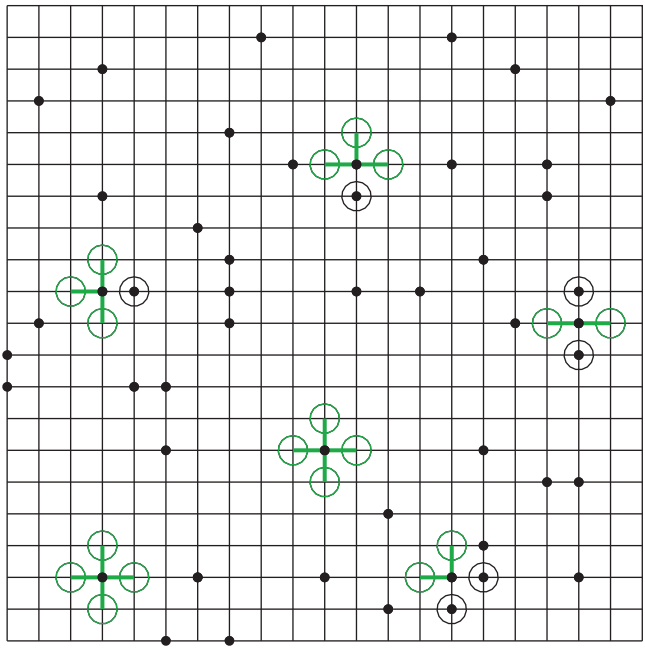

Figure D.6. Displacement rules for random walks on a lattice in which walkers are not allowed to move to a node already occupied by another walker. Here, 50 walkers (black solid dots) are distributed on a 20×20 lattice. For a subset of 6 walkers, the four target nodes are indicated by green circles, and the allowed moves by thick green line segments.

flocking forces in the flocking simulations of chapter 10). This becomes computationally prohibitive for very large N. There exist strategies and algorithms to reduce this number, but they are too complex (!) to get into even this book.

Another, related attractive feature of random walks on lattices is the possibility of accommodating simplified representations of "collisions" between two walkers. The idea is illustrated in figure D.6. Often, at a given temporal iteration, only a small fraction of lattice nodes are occupied by random walkers (black

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

solid dots). Every one of these would normally take its next step to one of the four possible positions indicated by the stencil of four green circles centered on each walker, as indicated in the figure for just six walkers. The idea is to void the step if it would land the walker on a lattice node already occupied by another walker; in such a situation, the walker remains on their node until a new random step is attempted at the next temporal iteration. The allowed steps for the six selected walkers in figure D.6 are indicated by the thick green line segments. The fact that walkers cannot move to an occupied node represents a form of collision, since two walkers on neighboring nodes cannot cross but instead tend to move away from each other, in a statistical sense. The flow of fluids can be simulated in this manner. Chapter 8 of Wolfram's book on cellular automata (cited at the end of chapter 2) presents a few nice examples. See also the Wikipedia pages on *lattice gas automaton* and *lattice Boltzmann methods* for entry points into the technical literature:

http://en.wikipedia.org/wiki/Lattice_gas_automaton;
http://en.wikipedia.org/wiki/Lattice_Boltzmann_methods.

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms

This content downloaded from
152.46.28.190 on Fri, 08 May 2026 09:47:12 UTC
All use subject to https://about.jstor.org/terms